Agent & MAS Course Assignment

Running, Improving, and Comparing a Multi-Agent Research System

Topic: *LLM-assisted PLC code generation and industrial automation programming*

mianma05

October 18, 2026

摘要

This report documents two tasks. In Task 1 the repository `catDforD/LLM_MAS_AutoGen` (a three/four-agent research assistant built on Microsoft AutoGen) was cloned, run locally, and its execution logs were interpreted. In Task 2 the system was extended with a PLC code-generation and deterministic verification loop, run on the same topic, and compared with the original. All code, screenshots, and logs are included in the repository.

1 Task 1 — Running the Original MAS

1.1 Overview

The original MAS converts a research topic into a short cited report. The `basic.py` version uses a fixed pipeline `PlannerAgent` → `SearchAgent` → `WriterAgent` on a `RoundRobinGroupChat`. The `advanced.py` version adds a `ReviewerAgent` and a one-revision loop on a `SelectorGroupChat` with a deterministic `choose_next` router. The single real tool, `search_literature`, queries a bundled course source library and the Crossref API.

1.2 Run results

Both versions were run with the PLC topic using the `deepseek-chat` model. The basic run produced a five-section report with citations `[S1]`–`[S12]`; the advanced run triggered a review–revision cycle in which the reviewer found citation problems and the writer revised the report. Figure 1 and Figure 2 show the run output.

1.3 Execution-log interpretation

The trace (`trace.jsonl`) records every event. Three observations matter:

```
[Task 1] basic.py run (Planner -> Search -> Writer)
D:\国际化课程作业\MAS\LLM_MAS_AutoGen\.venv\Lib\site-packages\autogen_agentchat\agents\_assistant_agent.py:1109: UserWarning: Resolved model mismatch: deepseek-chat != deepseek-flash. Model mapping in autogen_ext.models.openai may be incorrect. Set the model to deepseek-flash to enhance token/cost estimation and suppress this warning.
  model_result = await model_client.create(
[user] 请围绕以下主题完成一次小型技术调研：LLM-assisted PLC code generation and industrial automation programming 要求：优先使用课程资料库和实时检索结果；报告面向本科生，避免堆砌术语。
[PlannerAgent] 我会把主题拆成 3 个检索问题，并给出给 SearchAgent 的关键词清单。  **检索问题 1：LLM 在 PLC 编程中的典型用法与能力边界** - 关注：LLM 辅助生成梯形图/结构化文本（ST）、代码补全、注释与文档生成、从自然语言需求到 PLC 代码的转换；哪些环节已较成熟、哪些仍不可靠。 - 面向本科生的解释点：PLC 编程的基本范式（梯形图、ST、功能块）与 LLM 介入的位置。  **检索问题 2：工业自动化场景下的安全、可靠性与验证方法** - 关注：LLM 生成 PLC 代码的错误类型（语法
[WriterAgent] **LLM 辅助 PLC 代码生成与工业自动化编程：技术小报告**  **一、问题定义**  PLC（可编程逻辑控制器）是工业自动化的核心控制设备，其编程主要遵循 IEC 61131-3 标准，常用梯形图（LD）和结构化文本（ST）两种语言 [S7]。近年来，大语言模型（LLM）在通用软件开发中展现出代码生成能力，研究者开始探索将其引入 PLC 编程：能否让 LLM 根据自然语言需求生成 PLC 代码，并在工业场景中安全使用？本报告围绕“LLM 在 PLC 编程中的用法、能力边界与验证方法”展开，面向本科生梳理现有

报告已保存：D:\国际化课程作业\MAS\LLM_MAS_AutoGen\outputs\basic_20261018_104714\report.md
消息轨迹已保存：D:\国际化课程作业\MAS\LLM_MAS_AutoGen\outputs\basic_20261018_104714\trace.jsonl
```

图 1: Task 1 — `basic.py` run (Planner → Search → Writer).

```
[Task 1] advanced.py run (with ReviewerAgent revision loop)
D:\国际化课程作业\MAS\LLM_MAS_AutoGen\.venv\Lib\site-packages\autogen_agentchat\agents\_assistant_agent.py:1109: UserWarning: Resolved model mismatch: deepseek-chat != deepseek-flash. Model mapping in autogen_ext.models.openai may be incorrect. Set the model to deepseek-flash to enhance token/cost estimation and suppress this warning.
  model_result = await model_client.create(
[user] 请围绕以下主题完成一次小型技术调研：LLM-assisted PLC code generation and industrial automation programming 要求：只根据检索到的证据写作，面向本科生，保留引用编号。
[PlannerAgent] 我会把主题拆成 3 个具体问题，并给出检索关键词。  **问题 1：LLM 在 PLC 代码生成中的现有方法与能力边界** - 具体问题：现有研究如何用 LLM 生成 PLC 代码（如梯形图、结构化文本、FBD）？采用哪些技术路线（提示工程、微调、RAG、多智能体、形式化验证结合）？在语法正确性、逻辑正确性、可编译性上的表现如何？ - 检索关键词：  - "LLM PLC code generation"  - "large language model programmable logic controlle
[WriterAgent] # LLM 辅助 PLC 代码生成与工业自动化编程：技术调研小报告  ## 1. 问题定义  PLC（可编程逻辑控制器）是工业自动化的核心控制设备，其编程通常遵循 IEC 61131-3 标准，涉及梯形图、结构化文本、功能块图等多种语言，并依赖 CODESYS、TIA Portal 等工程环境 [S8][S11]。近年来，LLM 在通用代码生成上表现突出，研究者开始探索其能否用于 PLC 代码生成，以及如何嵌入“需求→设计→编码→测试→调试”的工程流程。本报告聚焦三个问题：现有 LLM 生成 PLC 代码的方法与能
[ReviewerAgent] 审阅意见（最多 3 条）：  1. **引用与证据匹配问题：S8/S9/S10 不应作为 CODESYS/IEC 61131-3 工程背景的可靠支撑。** 这三条 Crossref 记录标题几乎相同、作者不同、年份标为 2026，且无摘要，明显是可疑/重复元数据（可能是同一本书的多个抓取条目）。报告第 1 节用 [S8] 支撑“PLC 编程遵循 IEC 61131-3、涉及梯形图/ST/FBD、依赖 CODESYS/TIA Portal”，第 2 节用 [S8][S11] 支撑工程实践背景——但 [S8] 本身无法
[WriterAgent] # LLM 辅助 PLC 代码生成与工业自动化编程：技术调研小报告（修订稿）  ## 1. 问题定义  PLC（可编程逻辑控制器）是工业自动化的核心控制设备。据检索到的 CODESYS 教材条目与 TIA Portal 环境文献标题所示，PLC 编程涉及 IEC 61131-3 标准及 CODESYS、TIA Portal 等工程环境 [S8][S11]；但这些条目均无摘要，其具体内容有待核实，本报告不将其作为事实性依据。近年来，LLM 在通用代码生成上表现突出，研究者开始探索其能否用于 PLC 代码生成，以及如何
[ReviewerAgent] 审阅意见：  1. **S6（GPTSwarm）的引用来源需确认。** 修订稿第 2、3、5 节仍引用 [S6]，但 S6 只出现在第一次检索结果中，后续两次检索结果列表（含最终“课程资料库”汇总）均未包含 S6。请确认 S6 是否属于本次检索证据链；若无法确认，应删除 [S6] 相关表述（“GPTSwarm 把语言 Agent 建模为可优化的图”及第 3、5 节引用），或明确标注其来源。这是引用可追溯性问题，需修正。  2. **第 3 节“形式化方法……提供最强保证，但成本高、覆盖面窄”仍属无证据外推。** [

报告已保存：D:\国际化课程作业\MAS\LLM_MAS_AutoGen\outputs\advanced_20261018_104759\report.md
审阅意见已保存：D:\国际化课程作业\MAS\LLM_MAS_AutoGen\outputs\advanced_20261018_104759\review.md
修订次数：1
```

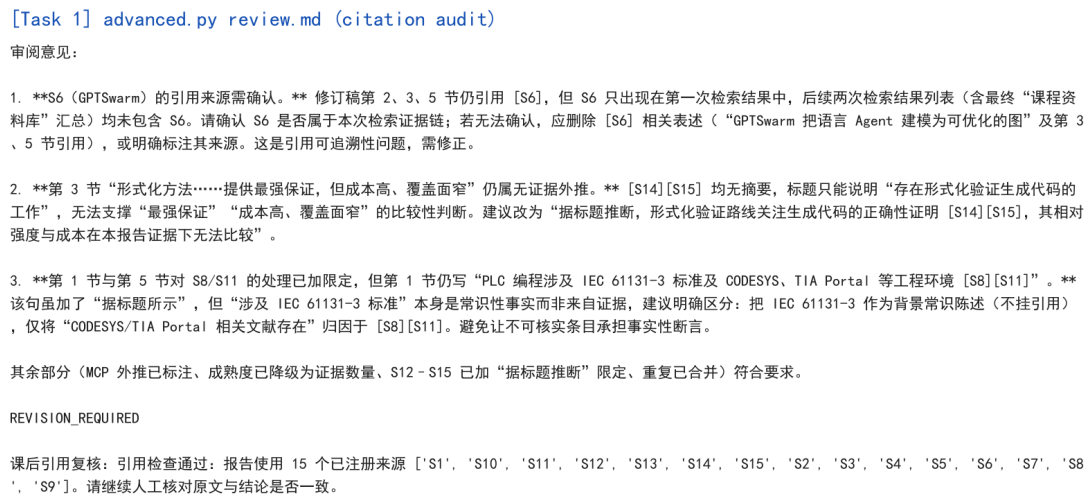
图 2: Task 1 — `advanced.py` run with the ReviewerAgent revision loop.

1. **Real tool calls.** `SearchAgent` issued three `search_literature` calls with different query strings. Each returned an evidence list with `[S#]` numbers, linking the tool result to the later report.

2. **Agent-to-agent message passing.** The planner's keywords flow into the searcher's tool calls, whose results are forwarded as `ToolCallSummaryMessage`s to the writer, which cites them.

3. **Termination and evidence traceability.** The basic run stops on the `BASIC_REPORT_DONE` sentinel; the advanced run stops on `REVIEW_APPROVED`/ `REVISION_REQUIRED` with a one-revision cap. The `evidence_registry.json` maps every cited `[S#]` back to a real source, and `audit_citations` verifies it.

Figure 3 shows the reviewer's feedback and the deterministic citation audit.


[Task 1] advanced.py review.md (citation audit)

审阅意见：

1. **S6（GPTSwarm）的引用来源需确认。** 修订稿第 2、3、5 节仍引用 [S6]，但 S6 只出现在第一次检索结果中，后续两次检索结果列表（含最终“课程资料库”汇总）均未包含 S6。请确认 S6 是否属于本次检索证据链；若无法确认，应删除 [S6] 相关表述（“GPTSwarm 把语言 Agent 建模为可优化的图”及第 3、5 节引用），或明确标注其来源。这是引用可追溯性问题，需修正。

2. **第 3 节“形式化方法……提供最强保证，但成本高、覆盖面窄”仍属无证据外推。** [S14][S15] 均无摘要，标题只能说明“存在形式化验证生成代码的工作”，无法支撑“最强保证”“成本高、覆盖面窄”的比较性判断。建议改为“据标题推断，形式化验证路线关注生成代码的正确性证明 [S14][S15]，其相对强度与成本在本报告证据下无法比较”。

3. **第 1 节与第 5 节对 S8/S11 的处理已加限定，但第 1 节仍写“PLC 编程涉及 IEC 61131-3 标准及 CODESYS、TIA Portal 等工程环境 [S8][S11]”。**该句虽加了“据标题所示”，但“涉及 IEC 61131-3 标准”本身是常识性事实而非来自证据，建议明确区分：把 IEC 61131-3 作为背景常识陈述（不挂引用），仅将“CODESYS/TIA Portal 相关文献存在”归因于 [S8][S11]。避免让不可核实条目承担事实性断言。

其余部分（MCP 外推已标注、成熟度已降级为证据数量、S12–S15 已加“据标题推断”限定、重复已合并）符合要求。

REVISION_REQUIRED

课后引用复核：引用检查通过：报告使用 15 个已注册来源 ['S1', 'S10', 'S11', 'S12', 'S13', 'S14', 'S15', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9']。请继续人工核对原文与结论是否一致。

图 3: Task 1 — `advanced.py review.md`: reviewer feedback and citation audit.

2 Task 2 — Improvement: PLC Code Generation & Verification

2.1 Improvement idea

The original system only *describes* how an LLM-based MAS can generate PLC code. The improvement makes it *actually generate and verify* PLC code: after the research stage, a new code stage runs `CodeGeneratorAgent` → `CodeVerifierAgent`, where the verifier calls a new deterministic tool `check_st_code` and the result drives a generate–verify–revise loop.

2.2 New components

- **CodeGeneratorAgent** — generates a demo IEC 61131-3 Structured Text (ST) program (motor start/stop with an emergency-stop interlock and a `TON` timer).

- **CodeVerifierAgent** — calls `check_st_code` and reports the verdict (`reflect_on_tool_use=False`, like the original searcher, for determinism).
- **check_st_code** (in `plc_tools.py`) — a real, deterministic tool: it strips comments and checks keyword pairing (`IF/END_IF`, `PROGRAM/END_PROGRAM`, …), bracket balance, and the misuse of `=` where ST requires `:=`. It returns `PASS`/`FAIL`.
- **Deterministic routing** — the code-stage router inspects the tool result directly: `FAIL` (and revision budget left) → back to the generator; `PASS` → stop.

This satisfies the Task 2 requirements: new roles and architecture, and at least one real tool call (`check_st_code`) whose result is used later to drive the revision decision.

2.3 Run results

The improved MAS was run on the same topic. Figure 4 shows the two-stage run. The generated code passed verification on the first attempt (`code_verification_pass = true`).



图 4: Task 2 — `improved.py` run (research stage + code stage).

[Task 2] improved.py generated_code.st (verified PASS)

```
(*
 * □□□□□□□□IEC 61131-3 □□□□□□ ST□
 * □□□
 *   1) □□□□ xStart □□□□□□□□□□ xStop □□□□□□□□□□
 *   2) □□ xEStop □□□□□□□□□□□□□□□□□□□□□□□□□□□□
 *   3) □□□□□□□□□□□□□□□□□□□□□□TON□□□□□□□□
 *   4) □□□□□□□□□□□ xReset □□□□□□□□□□□□□□□□
 *)

PROGRAM MotorControl
VAR
    (* □□ *)
    xStart      : BOOL := FALSE;   (* □□□□□□□□ *)
    xStop       : BOOL := FALSE;   (* □□□□□□□□ *)
    xEStop      : BOOL := TRUE;    (* □□□□□□□TRUE=□□□□□□□ *)
    xReset      : BOOL := FALSE;   (* □□□□□□□□□□ *)

    (* □□ *)
    xMotor      : BOOL := FALSE;   (* □□□□□□□□ *)
    xFaultLamp  : BOOL := FALSE;   (* □□□□□□ *)

    (* □□□□ *)
    xRunCmd     : BOOL := FALSE;   (* □□□□□□□□□□ *)
    xFault      : BOOL := FALSE;   (* □□□□□ *)
    xSafetyOK   : BOOL := FALSE;   (* □□□□□□□ *)

    (* □□□ *)
    tonStartDelay : TON;           (* □□□□□□□ *)
END_VAR

(* ---------- 1. □□□□□□ ---------- *)
(* □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ *)
xSafetyOK := xEStop AND NOT xFault;

(* ---------- 2. □□□□□□□ ---------- *)
IF NOT xEStop THEN
    (* □□□□□□□□□□□□ *)
    xFault := TRUE;
ELSIF xReset AND xEStop THEN
    (* □□□□□□□□□□□□□□□□□ *)
    xFault := FALSE;
END_IF;

(* ---------- 3. □□□□□□□□□□/□□□ ---------- *)
IF xFault THEN
    (* □□□□□□□□□□□□□□□□ *)
    xRunCmd := FALSE;
ELSIF xStart AND xSafetyOK THEN
    xRunCmd := TRUE;
ELSIF xStop THEN
    xRunCmd := FALSE;
END_IF;

(* ---------- 4. □□□□□□□□TON□ ---------- *)
(* □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ *)
tonStartDelay(IN := xRunCmd AND xSafetyOK, PT := T#2S);

(* ---------- 5. □□□□ ---------- *)
IF xFault THEN
    xMotor := FALSE;
ELSIF tonStartDelay.Q THEN
    xMotor := TRUE;
ELSE
    xMotor := FALSE;
END_IF;

(* ---------- 6. □□□□ ---------- *)
xFaultLamp := xFault;

END_PROGRAM
```

图 5: Task 2 — the verified IEC 61131-3 ST program (`generated_code.st`).

[Task 2] improved.py verification.md

Agent 工具校验（check_st_code）：
第 1 次工具调用结果：
PASS
- 未发现明显问题：关键字配对、括号、赋值运算符均正常。

确定性复核（check_st_code）：
PASS
- 未发现明显问题：关键字配对、括号、赋值运算符均正常。

图 6: Task 2 — `verification.md`: the `check_st_code` result and deterministic re-check, both `PASS`.

3 Comparison of the Original and Improved MAS

Dimension	**Original (`advanced.py`)**	**Improved (`improved.py`)**
Roles	Planner, Search, Writer, Reviewer	+ CodeGenerator, CodeVerifier
Output	cited research report	research report + verified ST code
Tools	`search_literature`, `audit_citations`	+ `check_st_code`
Workflow	one research revision loop	research loop + code generate–verify loop
Reliability	citation audit, max turns	+ deterministic code verification
On-topic value	describes PLC-code MAS	also produces & checks PLC code

The improvement trades a small amount of extra runtime for a much more concrete demonstration: the MAS now produces and independently verifies domain code instead of only describing it. The verification is deterministic, so the system does not rely on the LLM to judge its own output — a reliability property that is especially important for industrial-control code.

4 Conclusion

Both tasks were completed. The original MAS was run and its logs interpreted (Task 1); a PLC code-generation and verification loop was added, run on the same topic, and compared with the original (Task 2). The repository contains the full code (with `README.md`), the two research reports under `reports/`, and run screenshots under `screenshots/`.